VDK/TLM Creation

Rapid Training

X-2025.06

Module 2: Lab A

Create the Model Skeleton of an SPI Controller

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview 2
Task-1: Create a new TLMC Project 2
Task-2: Add Model Parameters 4
Task-3: Add Model Interfaces 6
Task-4: Add Memory Map 9
Task-5: Define Interrupt Behavior 14
What's Next 14

Lab Overview

Your task for this module is to create and test a model of a Serial Protocol Interface (SPI) controller peripheral using the TLM Creation perspective of Virtualizer Studio that comes with a broad set of productivity features, including model versioning, continuous assignments, reuse objects, or protocol engines.

Please refer to the SPI Controller Specification for interface and behavior of this training example model.

In this first lab, you will create the skeleton of the SPI controller i.e. create a new project for the new model, add parameters for behavior and versioning as well as interfaces, and a memory map.

Task-1: Create a new TLMC Project

***Motivation**: This task describes the steps to create the new TLMC model project and fill the basic information.*

1. Start Virtualizer Studio.

   Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

   - `% source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)`

     `% source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)`

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

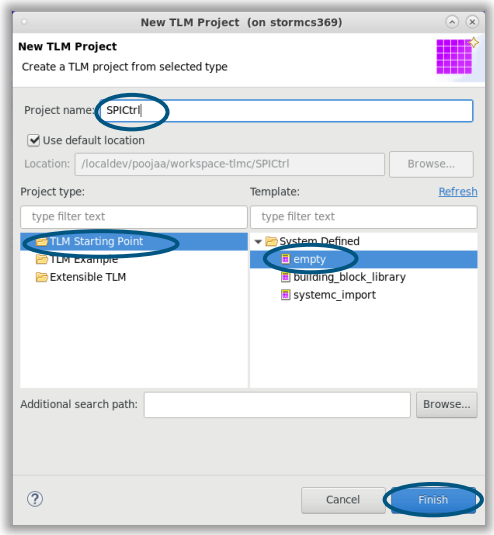
   - `% vs&`
   - `% cd <TrainingData>/module_2`

     The tool opens and displays the Workspace Launcher window.

2. Specify your workspace location, where you want to save your projects and click ***Launch*** on the Workspace Launcher window.

3. Specify the following in the ***New TLM Project*** dialog:

   - Type `SPICtrl` in the ***Project name*** field,
   - Select the `TLM Starting Point` option in the ***Project type*** field,
   - Select the `empty` Model Template from the available options.

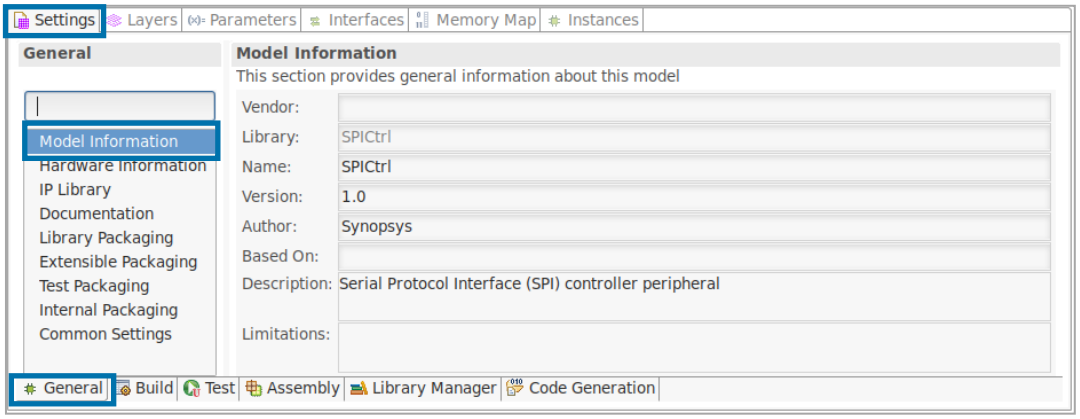


- Click ***Finish***.

  When prompted, click ***Open Perspective*** to open the ***TLM Creation*** perspective. The project opens in the ***TLM Creation*** perspective, and the ***Settings*** tab of the model editor displays.

4. Review the prepopulated fields and specify the following details in the ***General*** page of the tab.

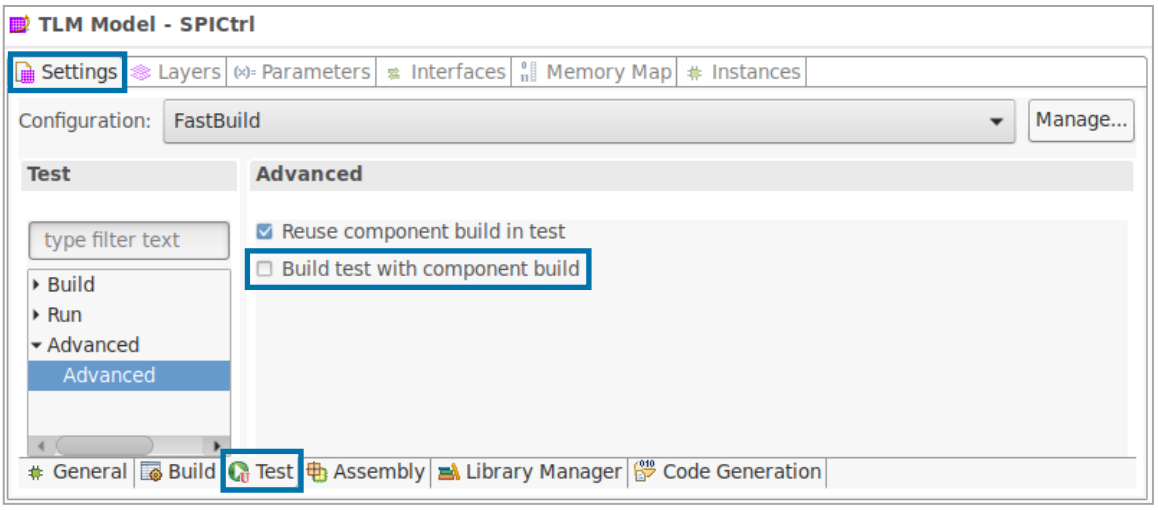
- ***Name***: SPICtrl (Project name)
- ***Version***: *1.0*
- ***Author:*** *Synopsys*
- ***Description:*** *Serial Protocol Interface (SPI) controller peripheral*



Leave the other fields empty or with their default values for now.

5. Next, switch to the ***Test*** page of the ***Settings*** tab. Open the ***Advanced*** settings under the ***Advanced*** head and click to uncheck the ***Build test with component build*** checkbox.

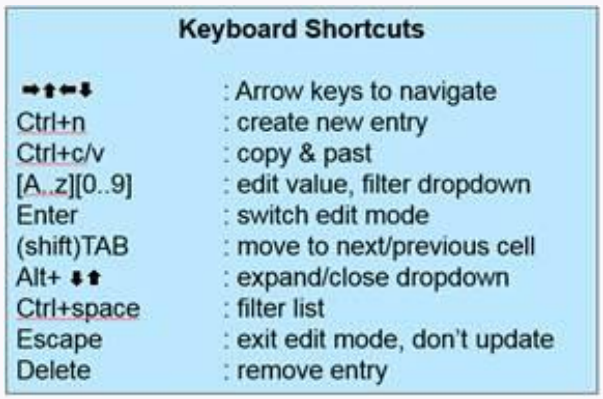
This will disable the compilation of the unit test (that we don't need yet) and speed up the overall build time.



6. Save the project using the ***Save*** button on the toolbar.

Task-2: Add Model Parameters

***Motivation**: This task describes the parameters to be added to the model. Notice the table-based format that allows to add and edit the parameters directly. The following commands/shortcuts can be used to navigate and edit all tables in TLM Creator:*



7. Go to the ***Parameters*** tab, click on the table and use <*CTRL-n*> or pick ***Add Parameter*** from the context menu, or use the ***Add*** icon ( ), or double-click to add a new entry on the table. You can use <*TAB*> to move to the next field in the table.

   Add the following parameters directly in the table:

   i. A parameter to enable/disable the check on the SPI clock communication:

   - ***Name***: `enable_check_clocks`
   - ***Type***: `bool`
   - ***Default Value***: `false`
   - ***Description***: Enable/disable the check on the SPI clock communication

   Leave other fields as is with their default value.

   ii. A parameter to identify the SPI when in slave mode:

   - ***Name***: `target_id`
   - ***Type***: `unsigned int`
   - ***Default Value***: `0`
   - ***Description***: Identify SPI when in input mode

   Leave other fields as is with their default value.

   iii. A parameter to configure the size of the internal FIFOs:

   - ***Name***: `fifo_size`
   - ***Type***: `unsigned int`
   - ***Default Value***: `5`

- ***Description***: Size of the internal FIFOs

Leave other fields as is with their default value.

8. Save and build the project.

Task-3: Add Model Interfaces

***Motivation**: This task describes the interfaces to be added to the model. Notice the table-based format that allows to add and edit the interfaces directly on the table. You have the option to define a Protocol, Role and Category per interface. This will populate automatically the preferred interface type and it will add protocol engines for those interfaces.*

9. Go to the ***Interfaces*** tab, click on the table and use <CTRL-n> or pick ***Add Interface*** from the context menu, or use the ***Add*** icon ( ), or double-click to add a new entry on the table. You can use <*TAB*> to move to the next field in the table.

   Add the following interfaces directly on the table:

   i. A bus target interface with

      - ***Name**:* `bus_reg_in`
      - ***Protocol:*** `TLM2`
      - ***Role**:* `target` (Category and Type will be defined automatically)
      - ***Mode**:* `LT`

   ii. A SPI serial interface with

      - ***Name**:* `Serial_IO`
      - ***Protocol**:* `SPI`
      - ***Role**:* `device`

   iii. An input reset interface with

      - ***Name***: `rst_in`
      - ***Protocol***: `Reset`
      - ***Role***: `input`

   iv. Two clock input ports, one for the SPI clock reference and the second one for the bus clock.

      - ***Name***: `spi_clk`, `clk_in`
      - ***Protocol**:* *`Clock`*
      - ***Role**:* *`input`*.

   v. A number of interrupts:

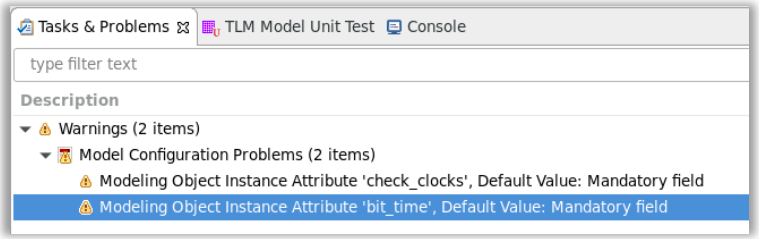
      - ***Name***: `tfuf_irq`, `tfff_irq`, `rfof_irq`, `rfdf_irq`
      - ***Protocol***: `Interrupt`
      - ***Role***: `driver`

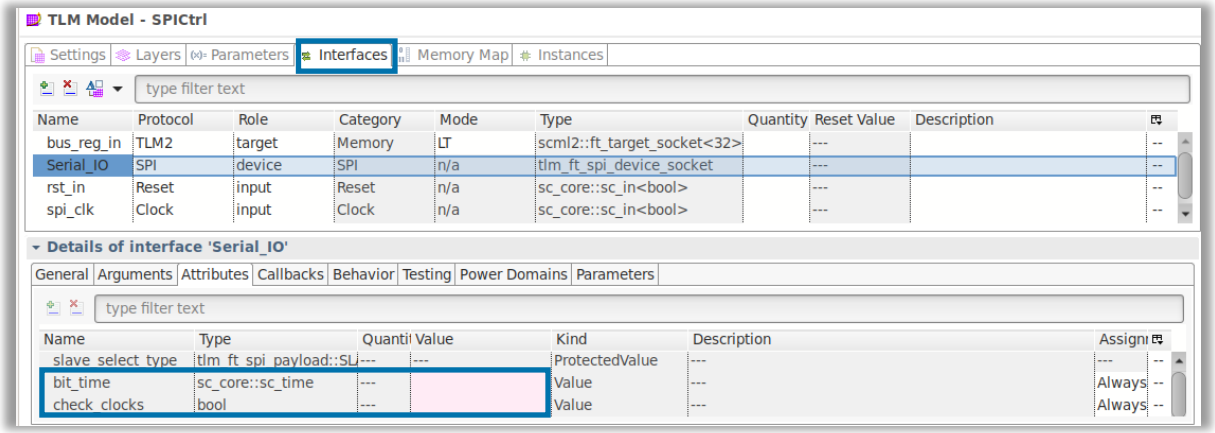
Note: You can right-click and select the ***Copy*** and ***Paste*** options to copy new entries with the same properties.

We have defined all interface elements of the SPI controller now. Look at the ***Tasks & Problems*** tab. You will see some open issues with the definitions as follows.

10. To know about the first warning*bit_time* attribute, switch to Interfaces tab.

    To display the attributes of the `Serial_IO` interface, expand the ***Details of interface Serial_IO*** to highlight the root cause of the warning – in this case a missing entry for a mandatory attribute.



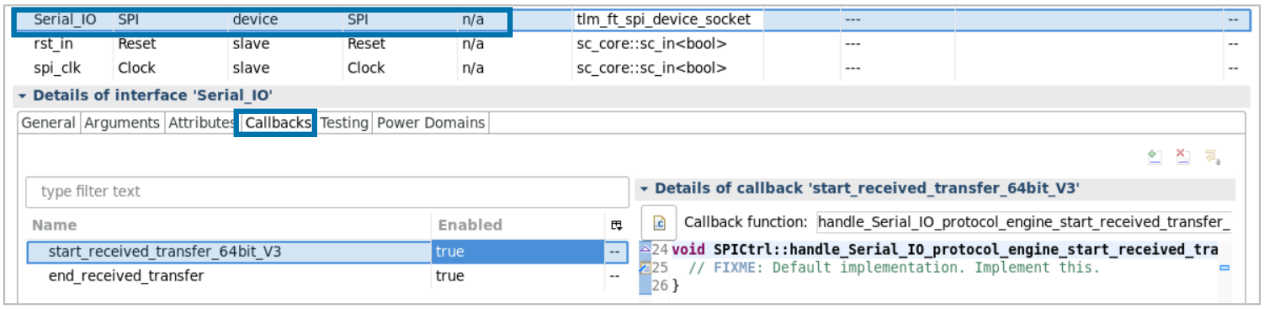


11. For the `Serial_IO` port, make the following changes.

    i. In the ***Attributes*** tab, assign `sc_core::sc_time(1, SC_US)` as ***Value*** to the attribute *bit_time,* and the parameter *enable_check_clocks* as ***Value*** to the attribute *check_clocks*.

    ii. In the ***Callbacks*** tab, you will notice that there are two mandatory callbacks that belong to the protocol engine connected to the socket. Note that the callbacks are mandatory because they cannot be disabled. If you click on a callback, you can see its name in the right panel, and that gets generated when you save your project. Moreover, in the peephole editor below, you can see the code area, where the source code of the callback is shown.

    Note: You need to save the project for the tool to generate the callback code.



    You can edit the callback functionality directly in the peephole editor or switch to the source code file by clicking on the ***Source Code Editor*** icon ( ) next to the callback name. The two views are kept continuously in sync. For now, we will leave the callbacks unimplemented.

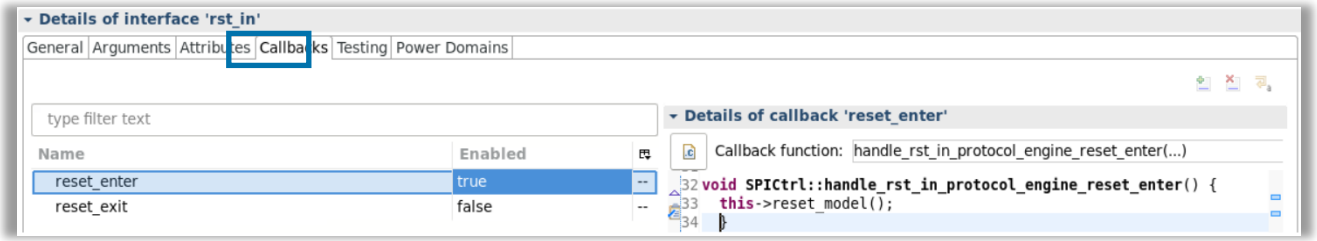
12. Next, select the `rst_in` reset interface

    i. In the ***Attributes*** tab of the ***Details*** section, make sure that the attribute *active_level* is set to value `false`.

    ii. In the ***Callbacks*** tab, change the status of the *reset_enter* callback to `true` in the ***Enabled*** column. Save the project to generate the source code again. The generated source code of the callback gets displayed.

    iii. Add the following functionality to the *reset_enter* callback:

    ```
    this->reset_model();
    ```



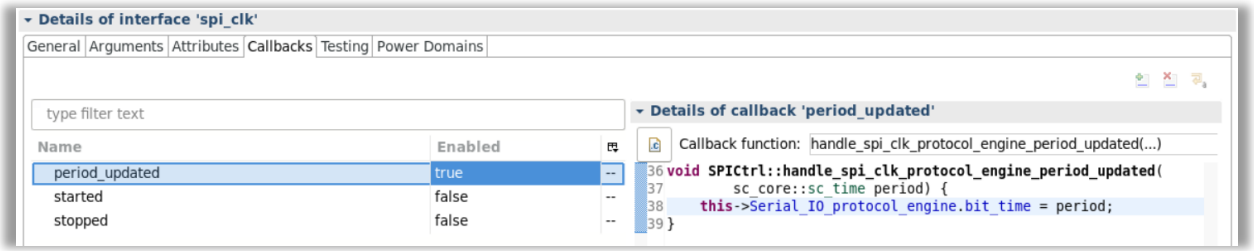
    iv. Remove the `FIXME` comment.

    v. Switch to the source code file clicking on the ***Source Code Editor*** icon ( ) – note that the same code is now visible in the source code file as well.

13. Next, select the `spi_clk` clock interface and perform the following tasks.

    i. In the ***Callbacks*** tab, enable the callback *period_updated*. Associate the clock period to the SPI bit time in the callback implementation, as described below:

```
this->Serial_IO_protocol_engine.bit_time = period;
```



    ii. Save and build the project.

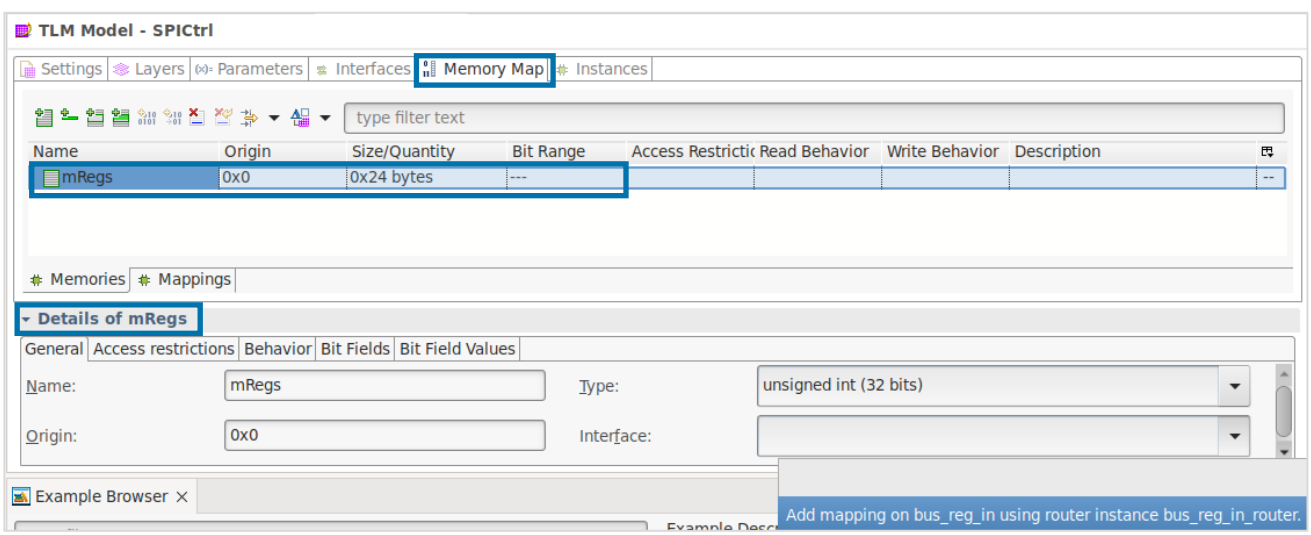
Task-4: Add Memory Map

***Motivation**: This task describes the memories, registers and bitfields to be added to the model. Notice the table-based format that allows you to add and edit all the memory map elements directly on the table, including bitfields.*

14. Switch to the ***Memory Map*** tab, click on the table and use *<CTRL-n>*, or pick ***Add Memory*** from the context menu, or use the ***Add Memory*** icon ( ), or double-click to add a new entry on the table. You can use *<TAB>* to move to the next field in the table.

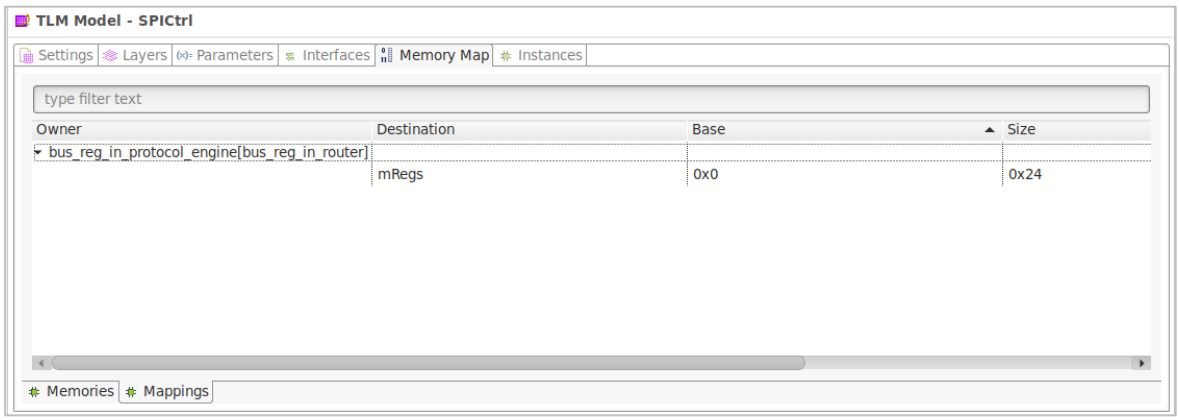
    Configure the new memory element as follows:

    - ***Name***: `mRegs`
    - ***Size***: `0x24`
    - In the ***Details for 'mRegs'*** panel, focus on the select the ***Interface*** field and select the *`Add mapping on bus_reg_in using router`* option.

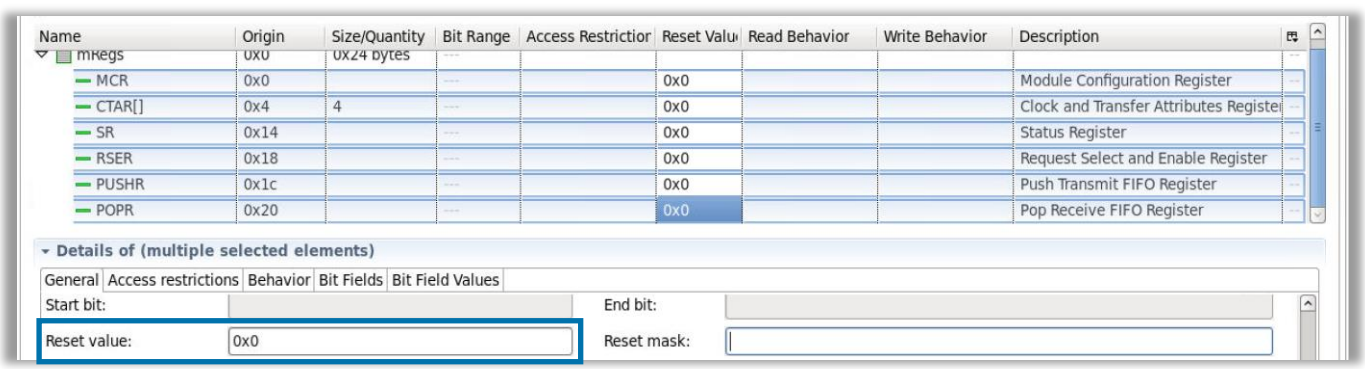


    - Select the default value *`0x0`* as the mapping base address and click ***OK*** to proceed.

The mapping gets added and can be viewed from the ***Mappings*** tab of the Details view.



15. Next, add the registers using `<CTRL-n><CTRL-r>`, or right-click and select ***Add Register*** from the context menu, or use the ***Add Register*** icon ( ). Add the following registers:
    i. ***Name***: `MCR` – ***Description***: `Module Configuration Register`
    ii. ***Name***: `CTAR` – ***Size/Quantity***: `4` – ***Description***: `Clock and Transfer Attributes Register`
    iii. ***Name***: `SR` – ***Description***: `Status Register`
    iv. ***Name***: `RSER` – ***Description***: `Request Select and Enable Register`
    v. ***Name***: `PUSHR` – ***Description***: `Push Transmit FIFO Register`
    vi. ***Name***: `POPR` – ***Description***: `Pop Receive FIFO Register`
16. Display the ***Reset Value*** column in the table by clicking the ***Customize Table*** icon ( ) on the top-right of the table header and selecting the checkbox for ***Reset Value***. Then, select all registers in the table and in the ***Details for (multiple selected elements)*** view, set ***Reset Value*** to `0x0` to set all values in one go.



17. Next, add bitfields for the **`MCR`** register. You can do so by first selecting the `MCR` register, and then pressing the ***Add Bit Field*** button ( ) or by selecting ***Add Bitfield*** from the context menu, or by typing *<CTRL-n><CTRL-b>*. The bitfields to be added are:

i. ***Description***: Bit to configure the SPI in either Initiator (value 1) or Target mode (value 0)
Name: `MSTR`
Bit Range: `0:0` (just type `0` and click enter)

ii. ***Description***: Bit to enable the Peripheral Chip Select in Strobe mode (value 1)
Name: `PCSSE`
Bit Range: `6:6` (just type `6` and click enter)

iii. ***Description***: Bitfield to set the Underflow alarm level of the Transmit Fifo
Name: `UFL_TXF`
Bit Range: `8:11`

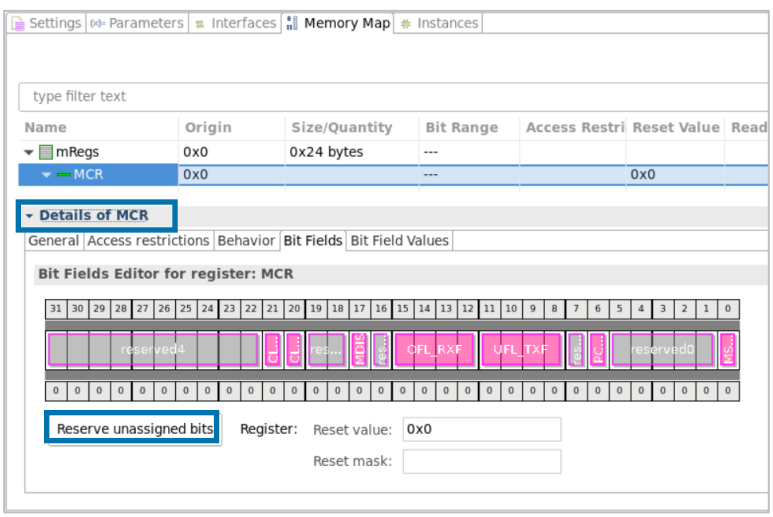
iv. ***Description***: Bitfield to set the Overflow alarm level of the Receive Fifo
Name: `OFL_RXF`
Bit Range: `12:15`

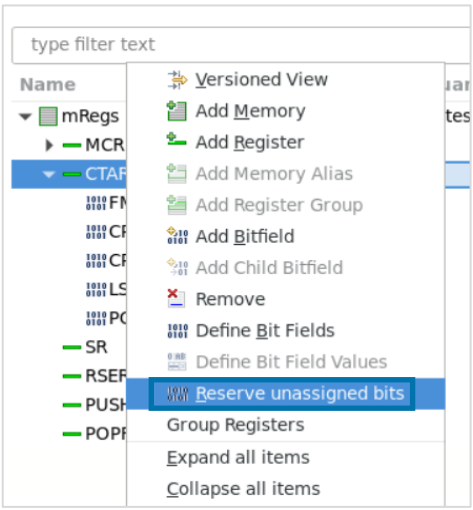
v. ***Description***: Bit to disable the module
Name: `MDIS`
Bit Range: `17:17` (just type `17` and click enter)

vi. ***Description***: Bit to clear/flush the TX FIFO
Name: `CLR_TXF`
Bit Range: `20:20` (just type `20` and click enter)

vii. ***Description***: Bit to clear/flush the RX FIFO
Name: `CLR_RXF`
Bit Range: `21:21` (just type `21` and click enter)

Reserve all other unassigned bits: Switch to the ***Details*** panel, select the ***Bit Fields*** tab and click on ***Reserve unassigned bits***, or select ***reserve unassigned bits*** from the context menu of the register.





18. Next, implement the bitfields of the **`CTAR`** registers as follows.

i. ***Description***: Bit to configure the Frame Size (bits transferred equals this value plus 1)
Name: `FMSZ`
Bit Range: `0:4`

ii. ***Description***: Bit to configure the Clock Polarity
Name: `CPOL`
Bit Range: `5:5`

iii. ***Description***: Bit to configure the Clock Phase
Name: `CPHA`
Bit Range: `6:6`

iv. ***Description***: Bit to configure the MSB or LSB transmit first option
Name: `LSBFE`
Bit Range: `7:7`

v. ***Description***: Clock Prescaler
Name: `PCSSCK`
Bit Range: `8:9`

vi. Reserve all the other fields of the register.

19. Now it is the turn for the bitfields of the status register **`SR`**, that also have some behavior/callbacks attached to them. The bitfields to add are the following:

i. ***Description***: Transfer Complete Flag
Name: `TCF`
Bit Range: `0:0`
Write Behavior: `Clear On Write 1`

ii. ***Description***: Transmit and Receive Status Flag
Name: `TXRXS`
Bit Range: `1:1`
Write Behavior: `Clear On Write 1`

iii. ***Description***: End of Queue Flag
Name: `EOQF`
Bit Range: `3:3`
Write Behavior: `Clear On Write 1`

iv. ***Description***: Transmit FIFO Underflow Flag
Name: `TFUF`
Bit Range: `4:4`
Write Behavior: `Clear On Write 1`

v. ***Description***: Transmit FIFO Fill Flag
Name: `TFFF`
Bit Range: `6:6`
Write Behavior: `Clear On Write 1`

vi. ***Description***: Receive FIFO Overflow Flag
Name: `RFOF`
Bit Range: `12:12`
Write Behavior: `Clear On Write 1`

vii. ***Description***: Receive FIFO Drain Flag
Name: `RFDF`
Bit Range: `14:14`
Write Behavior: `Clear On Write 1`

viii. Reserve all the other fields of the register.

20. Finally, implement the bitfields of the **`PUSHR`** register:

    i. ***Description***: Continuous Selection Format
       Name: `CONT`
       Bit Range: `0:0`

    ii. ***Description***: CTAS register selection
       Name: `CTAS`
       Bit Range: `1:3`

    iii. ***Description***: End Of Queue
       Name: `EOQ`
       Bit Range: `4:4`

    iv. ***Description***: Clear Transfer Counter
       Name: `CTCNT`
       Bit Range: `5:5`

    v. ***Description***: Peripheral Chip Select
       Name: `PCS`
       Bit Range: `8:15`

    vi. ***Description***: Data to transmit
       Name: `TXDATA`
       Bit Range: `16:31`

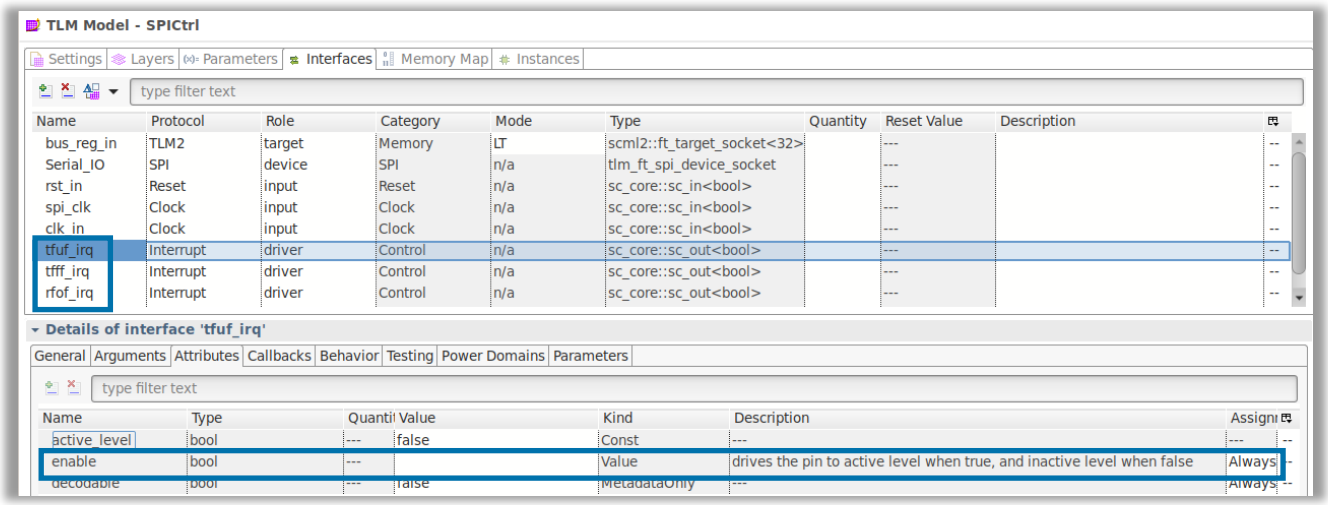
21. Save and build the project.

Task-5: Define Interrupt Behavior

***Motivation**: The interrupt behaviors of the model are directly linked to the status register bit fields that we defined in the previous task. Now we can use these bit fields to define the interrupt behavior using simple continuous assignments.*

22. Open the ***Interfaces*** tab and focus on the `enable` attribute setting for the interrupt ports in the ***Attributes*** tab for under the *Details* section.

    The *enable* attribute defines when the interrupt is triggered / enabled, i.e. it is a Boolean expression that is continuously being assigned to the interrupt port (***Assignment*** = `Always` means continuous assignment). The ***Value*** field for this attribute is currently empty.



    Assign the ***Value*** field for the `enable` attributes for the interrupts as follows:

    i. ***Interface***: `tfuf_irq`
       enable: `(SR.TFUF == 1)`

    ii. ***Interface***: `tfff_irq`
       enable: `(SR.TFFF == 1)`

    iii. ***Interface***: `rfof_irq`
       enable: `(SR.RFOF == 1)`

    iv. ***Interface***: `rfdf_irq`
       enable: `(SR.RFDF == 1)`

23. Save and build the project.

What's Next

In the next lab, we will implement the behavior of the SPI controller using reuse objects in the TLMC libraries.